



Design of an Open-Source Bridge Between Non-Coherent Burst-Based and Coherent Cache-Line-Based Memory Systems

Matheus Cavalcante

Integrated Systems Laboratory, ETH Zürich
Zürich, Switzerland
matheusd@iis.ee.ethz.ch

Fabian Schuiki

Integrated Systems Laboratory, ETH Zürich
Zürich, Switzerland
fschuiki@iis.ee.ethz.ch

Andreas Kurth

Integrated Systems Laboratory, ETH Zürich
Zürich, Switzerland
akurth@iis.ee.ethz.ch

Luca Benini

Integrated Systems Laboratory, ETH Zürich
Zürich, Switzerland
DEI, Università di Bologna
Bologna, Italy
lbenini@iis.ee.ethz.ch

ABSTRACT

In heterogeneous computer architectures, the serial part of an application is coupled with domain-specific accelerators that promise high computing throughput and efficiency across a wide range of applications. In such systems, the serial part of a program is executed on a Central Processing Unit (CPU) core optimized for single-thread performance, while parallel sections are offloaded to Programmable Manycore Accelerators (PMCA). This heterogeneity requires CPU cores and PMCA to share data in memory efficiently, although CPUs rely on a coherent memory system where data is transferred in cache lines, while PMCA are based on non-coherent scratchpad memories where data is transferred in bursts by DMA engines. In this paper, we tackle the challenges and hardware complexity of bridging the gap from a non-coherent, burst-based memory hierarchy to a coherent, cache-line-based one. We design and implement an open-source hardware module that reaches 97% peak throughput over a wide range of realistic linear algebra kernels and is suited for a wide spectrum of memory architectures. Implemented in a state-of-the-art 22 nm FD-SOI technology, our module bridges up to 650 Gbps at 130 fJ/bit and has a complexity of less than 1 kGE/Gbps.

CCS CONCEPTS

• **Computer systems organization** → **Heterogeneous (hybrid) systems**; System on a chip; *Interconnection architectures*.

KEYWORDS

Heterogeneous computing, memory sharing, AMBA, CHI, AXI.

ACM Reference Format:

Matheus Cavalcante, Andreas Kurth, Fabian Schuiki, and Luca Benini. 2020. Design of an Open-Source Bridge Between Non-Coherent Burst-Based and

Coherent Cache-Line-Based Memory Systems. In *17th ACM International Conference on Computing Frontiers (CF '20)*, May 11–13, 2020, Catania, Italy. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3387902.3392631>

1 INTRODUCTION

Heterogeneous computing systems combine general-purpose CPU cores with PMCA, promising to deliver high performance and energy efficiency across a wide range of applications [26]. In such systems, the serial part of a program is executed on a CPU core optimized for single-thread performance. Parallel sections are offloaded to PMCA, e.g., data-parallel architectures such as General-Purpose Graphics Processing Units (GPUs), DSP-like parallel processors, or other general-purpose manycores [7].

The non-uniform memory hierarchy poses a major challenge for data sharing in heterogeneous systems: the CPU relies on a coherent memory system, where data is transferred in cache lines, while PMCA typically have a hierarchy of Scratchpad Memories (SPMs) where data is transferred in memory bursts, usually managed by Direct Memory Access (DMA) engines [20]. There are two major options to deal with this *gap* in the memory system [6]. The most widely used option is to *evade* the gap, by sharing data at the level of non-coherent (e.g., off-chip) memory. This option has the advantage of offering full memory bandwidth to the PMCA without additional hardware but requires flushing all CPU caches on every offload from a CPU to PMCA. Due to the high amount of time and energy required to flush all caches, this option is only attractive for very coarse-grained acceleration. This model is used to share data between CPUs and the latest NVIDIA Volta GPUs [23].

The second option is to *bridge* the gap with a hardware unit that translates non-coherent transfers by the PMCA to a coherent interconnect. This approach alleviates the overhead required to share data between the computing units. For example, the Accelerator Coherency Port (ACP) by ARM [4, Section F4.1] allows on-chip accelerators to be attached to the coherent interconnect of ARM CPUs and the Coherent Accelerator Processor Interface (CAPI) by IBM allows off-chip accelerators connected via Peripheral Component Interconnect Express (PCIe) to participate in the system-wide coherence protocol as a peer of other caches in the system [31]. Promising results have been demonstrated with such approach, with coherence having a small performance impact given

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

CF '20, May 11–13, 2020, Catania, Italy

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7956-4/20/05...\$15.00

<https://doi.org/10.1145/3387902.3392631>

that the memory latency is kept low [29]. However, to the best of our knowledge, no work describes in detail the architecture and hardware design of a coherence-preserving bridge between CPUs and PMCAs.

In this work, we fill this knowledge gap for the industry-dominant Advanced Microcontroller Bus Architecture (AMBA) family of open standards for on-chip communication. We designed a bridge between a state-of-the-art multi-core cluster with internal communication based on the Advanced eXtensible Interface (AXI) protocol [4]—a high-performance burst-based protocol for non-coherent communication—to a host CPU based on the Coherent Hub Interface (CHI) protocol [3]—a credit-based protocol optimized for scalable, coherent cache line accesses. We analyze the requirements for the bridge, describe the resulting architecture, and evaluate its throughput, area, and energy consumption for a wide range of linear algebra kernels. The key contributions of this paper are:

- (1) The architectural definition and full design of an IO-coherent bridge between AMBA AXI and CHI protocols. The bridge will be open-sourced under a permissive license. In terms of performance, the bridge reaches 97% peak throughput over realistic linear algebra and stencil kernels.
- (2) A performance and energy efficiency exploration of our design with post-synthesis results extracted from GLOBALFOUNDRIES 22FDX Fully Depleted Silicon on Insulator (FD-SOI) technology.
- (3) An analysis of the performance degradation caused by CHI as a function of the memory latency, on memory-bound and compute-bound kernels running on the PMCA.

The rest of this paper is structured as follows. Section 2 briefly describes the system template and interfaces we are targeting. Section 3 discusses the challenges and the architecture of our bridge, whose results in terms of performance, power, area, and performance impact on a state-of-the-art streaming coprocessor are discussed in Section 4. Section 5 compares our contribution to related work, and Section 6 concludes.

2 BACKGROUND

2.1 Heterogeneous Architecture Template

Figure 1 shows the general template of the heterogeneous computers we target in this work. An application-class or server-class multi-core CPU with a cache hierarchy and coherent interconnects (with the CHI protocol, for instance) is the centerpiece of the computer. After the last-level interconnect and cache, the CPU interfaces the (off-chip) shared main memory. Attached to the coherent interconnects of the CPU through a bridge like the one designed in this work are PMCAs. The cache level at which a PMCA is attached depends on the granularity of the workload that PMCA accelerates. PMCAs are multi-to-many-core parallel architectures in themselves, featuring a hierarchy of software-managed SPMs, the outermost of which can even be off-chip depending on the bandwidth and size requirements of the PMCA. Data is moved in and out of SPMs by DMA engines through burst-based high-bandwidth on-chip interconnects (with the AXI protocol, for instance).

State-of-the-art examples that implement this architectural template range from warehouse-scale computers such as the Cray XC series [9]—using an Intel Xeon CPUs with either NVIDIA Tesla

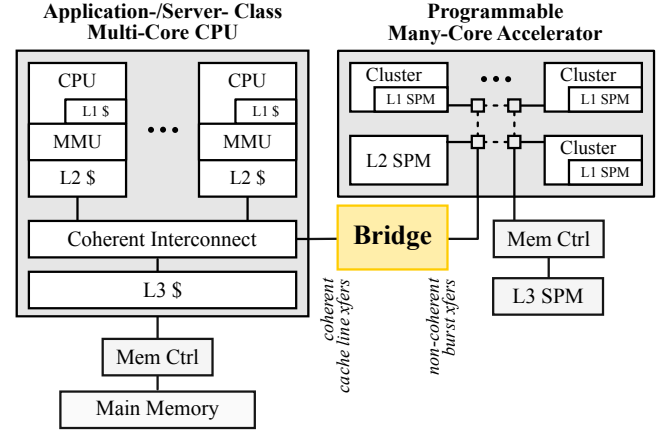


Figure 1: Template for heterogeneous computer architectures, with our bridge highlighted.

or Intel Xeon Phi accelerators—to heterogeneous SoCs such as the NVIDIA Tegra [22], Samsung Exynos [14], Qualcomm Snapdragon [27], and the Apple A [13], to research platforms such as Intel’s HARP [18] and HERO [19].

2.2 AMBA AXI and CHI Standards

The open AMBA standards by ARM are widely adopted throughout the industry for on-chip communication. In this work, we will focus on AXI5 [4], a high-performance burst-based protocol for non-coherent communication, and on the recent CHI [3], a cache-line-based protocol with credit-based flow control for scalable communication among many coherent agents.

2.2.1 Advanced eXtensible Interface (AXI). AXI employs burst-based transactions and supports multiple outstanding transactions in parallel, as well as transaction reordering. Communication is separated into two directions, read and write, and into dedicated channels for commands, data, and responses. As a result, AXI has five channels: AW for write commands, W for write data, B for write responses, AR for read commands, and R for read responses and data. An AXI transaction consists of a command followed by one or multiple data beats and a single response, on a write, or one or multiple data and response beats, on a read. Each transaction specifies an ID, and for every ID, multiple transactions can be active at the same time. Transactions are ordered accordingly to two rules: a transaction in one direction is not ordered with respect to any transaction in the other direction, and transactions in the same direction are ordered if both transactions share the ID. For any two ordered AXI transactions, all intermediary components must ensure that the requests reach the slave(s) in the order they were issued and that the master receives response(s) in the same order.

2.2.2 Coherent Hub Interface (CHI). CHI transactions work on a cache-line granularity, which can be up to 512 bits long. Unlike AXI, ordering between transactions is not enforced. In fact, CHI is not designed for ordering by default. CHI deploys a credit-based flow control: before transmitting one flit—the smallest unit recognized by the flow control method [11]—the Requester must have obtained

an L-Credit (Link Layer Credit), and the Receiver must guarantee to accept all flits for which it has issued L-Credits [3, Section 13.2]. If a Receiver accepts a request, it answers with either a Comp or a CompData response (respectively, for write and read operations). If a Receiver cannot complete a request, it can emit a RetryAck response to avoid blocking its REQ channel [3, Section 2.11]. Each RetryAck flit specifies that the Requester must wait until it has a certain P-Credit (Protocol Layer Credit) type, before retrying the offending request. P-Credits are issued via PCrdGrant messages.

Each CHI transaction has an 8-bit Transaction ID (TxnID), which must be unique, i.e., not reused by a Requester until it has received all responses associated with a transaction. The fixed number of unique transaction IDs limits the latency up to which any CHI Requester can fully utilize the bandwidth of the memory system: when the 256 unique TxnIDs of a Requester are exhausted, it must wait for the end of an active transaction before it can issue the next request. Unique IDs are a protocol limitation specific to CHI, not found at the AXI specification level, which leaves unique transaction IDs as an optional extension [4, Section E1.16]. Of course, the finite number of buffers in any real system will also cause memory latency to limit the sustainable throughput. But system architects can always dimension these buffers to fit the memory system, which is only possible up to a strict limit in CHI.

3 ARCHITECTURE

Figure 2 shows the architecture of our AXI-CHI bridge. AXI channels are regrouped on the left side, whereas CHI channels can be seen on the right side of the diagram. The CHI specification supports 128, 256, and 512 bit-wide buses [3, Chapter 2.10.4]. Our bridge targets a 512-bit memory interface.

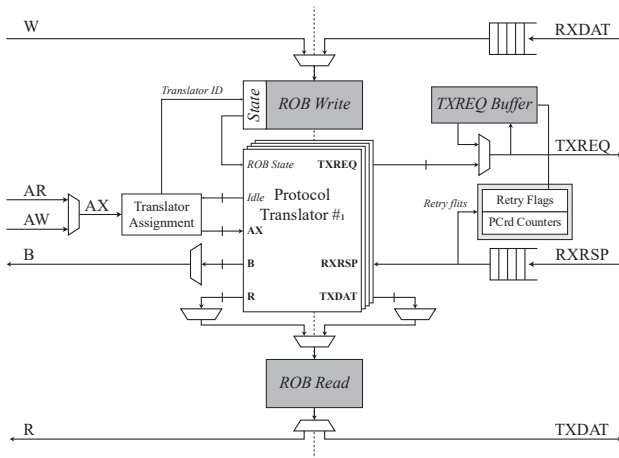


Figure 2: Block diagram of the AXI-CHI bridge.

Translating AXI to CHI commands is relatively straightforward, the protocol translators of Figure 2 being simple state machines sending requests and waiting for responses. The bridge has a few of these protocol translators working in parallel, each one translating one AXI transaction, and the CHI flits generated by these translators are multiplexed into the bridge's CHI interface. Incoming AXI transactions with the same AXID are handled by the same

protocol translator, to ensure the ordering of AXI transactions with the same AXID, as discussed in Section 2.2.1. Since the accelerator is an IO-coherent node, the bridge only needs to support a small subset of the CHI protocol, which is nevertheless enough to explore the system-wide coherence provided by the CHI. During the rest of this section, we analyze other features of the bridge further more interesting than the plain protocol translation.

3.1 Re-order Buffer

AXI bursts can be as long as 4 KiB [4, Section A3.4.1], while CHI transactions are 512 bits at their longest. Since the target CHI interface is also 512-bit wide, each CHI request flit corresponds to a single data flit. Atomic load and swap transactions are an exception, since they correspond to two CHI data flits, one in the RXDAT channel to read the old data, and one in the TXDAT channel to update data stored in the memory. Atomic compares are also an exception, triggering four data flits per request [3, Section 5.4].

Upon receiving an AXI burst, the bridge splits it up into a sequence of CHI requests, each with a unique TxnID. This is illustrated by the sequence diagram of Figure 3, where we use the same notation as the CHI specification [3]—here, Rn stands for the Request node, i.e., the bridge, and Hn stands for Home node, in this case, the CHI interconnect. Behind the Home node there are undepicted Slave nodes Sn receiving the read and write requests.

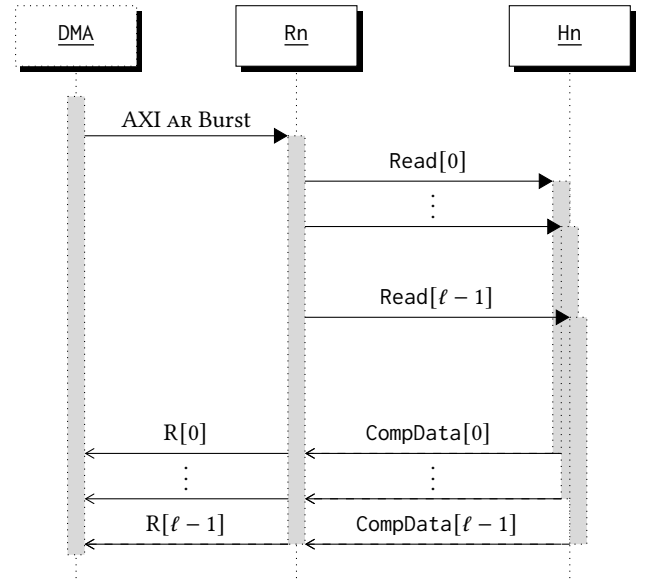


Figure 3: Sequence diagram for the translation of an AXI burst into CHI requests.

While Figure 3 might suggest that the CompData response associated with each Read request¹ comes back in order, the CHI

¹Read is not a valid CHI opcode, but rather stands for a family of opcodes with variations depending, among others, on the properties of the target memory region. Our bridge emits two types of Read messages: ReadNoSnp, for non-allocable memory regions, and ReadOnce, for allocable memory regions. The properties of the memory region are deduced from the ARCACHE field of the arriving AXI read transactions.

specification does not guarantee this constraint. Therefore, an important task of the bridge is to reorder the CHI CompData responses associated with an AR burst before forwarding them through the R channel. AXI ordering rules, as discussed in Section 2.2.1, are enforced by a Re-order Buffer (ROB), which is implemented as a 512-bit wide 1R1W Static Random-Access Memory (SRAM).

Another important distinction between AXI and CHI is that the read and write request channels of the latter are not independent. CHI provides a single request channel, TXREQ, which is used by both read and write requests. Moreover, since every CHI request corresponds to a single data flit, the bridge has a global peak throughput of 512 bit/cycle, shared between reads and writes. On the other hand, AXI has independent read and write channels, with a peak throughput of 512 bit/cycle *each*. The throughput of the bridge is adapted to the slowest interface, i.e., CHI. Both AR and AW requests are multiplexed into the TXREQ channel, and the ROB ports are shared between data read and data write channels. Since each port of the ROB has a peak throughput of 512 bit/cycle, a 1R1W ROB architecture is sufficient to saturate the CHI interface data channels, as illustrated in Figure 2.

As described in Section 2.2.2, CHI's requirement of unique TxnIDs puts a hard limit on the latency the bridge can support before suffering performance degradation. If the bridge is operating at its peak throughput, as soon as the memory latency surpasses 256 cycles, it will stall waiting for free transaction IDs. The uniqueness property of the TxnIDs is explored by the bridge, by using such IDs to index the data flits stored in the ROB, and to demultiplex the flits to the corresponding protocol translator.

3.2 Retry mechanism

To avoid blocking the bridge waiting for an acknowledgment or retry response from the Receiver, a buffer holds a copy of all in-flight request flits sent through the TXREQ channel, also indexed by their unique TxnIDs. The retry logic keeps track of which transactions received a RetryAck, the type of P-Credit these expect, and how many P-Credits of each type the bridge holds. Once the bridge is ready to retry a request, it is fetched from the TXREQ buffer, consuming the corresponding P-Credit during the process.

3.3 Configuration registers

Our bridge has some configuration registers, not depicted in Figure 2. These registers are used to set up the values of the srcid and tgtid fields of the outgoing TXREQ flits, which are later used by the Hn to route them. The srcid is the unique identifier of the bridge's CHI interface, and since all of its TXREQ flits are forwarded to the Hn, the outgoing flits have the unique identifier of the Home node as their tgtid. The configuration registers are written at configuration type by any debug interface available—in our case, we used a lightweight preexisting Advanced Peripheral Bus (APB) interface.

4 EVALUATION AND RESULTS

In this section, we analyze the implementation of several instances of our bridge, in terms of area, power, energy efficiency, and performance on realistic machine-learning benchmarks.

4.1 Methodology

In this work, we adopted the Heterogeneous Embedded Research Platform (HERO) [19] as our evaluation platform. Our PMCA is based on the open-source PULP cluster [28], with eight 32-bit RISC-V cores [15]. The PMCA has 256 KiB of L1 SPM, further divided into 16 banks that the cores can access within one cycle. The PMCA has a DMA capable of handling up to 16 outstanding AXI requests. The host processor is Ariane, an open-source RISC-V 64-bit core [33]. There is no L3 cache, and the coherent interconnect of Figure 1 is directly connected to the main memory.

We take five kernels out of the Polybench/ACC benchmark suite [17] to analyze the performance of our bridge. The suite includes a collection of codes for linear algebra, data-mining and stencils, from which we chose the kernels listed in Table 1. The kernels span both the linear algebra and the stencil domain. These commonly accelerated kernels span a wide range of memory access patterns and operational intensities [21]. The matrices are copied into and from the L1 SPM with the PMCA's DMA engine at the beginning and the end of the offload phase of the algorithms. All kernels are compiled with -O3 and no further optimization flags.

Table 1: Polybench/ACC kernels used in our benchmarks.

Kernel	Description
2mm	2 matrix multiply ($C \leftarrow A \cdot B$)
3mm	3 matrix multiply ($E \leftarrow A \cdot B; F \leftarrow C \cdot D; G \leftarrow E \cdot F$)
atax	Matrix transpose and vector multiplication
2dconv	2D convolution
gemm	Matrix multiply ($C \leftarrow \alpha A \cdot B + \beta C$)

The performance results were obtained via a cycle-accurate simulation of the Register Transfer Level (RTL) design with QuestaSim 2019.03. The testbench manages the main memory and provides a CHI port for the Device Under Test (DUT). More specifically, the CHI messages generated by the DUT are sent to a Slave node instantiated with Cadence's Verification IP 11.30.062, which besides managing the memory and generating the CHI response packets, also asserts the correctness of the CHI transactions. We assume that this CHI slave node supports a very large number of outstanding transactions, namely 256. Cadence's Verification IP allow us to set the latency of the CHI Slave node as we like.

The bridge was synthesized for GLOBALFOUNDRIES 22FDX FD-SOI technology using Synopsys Design Compiler 2019.03. Our area results are presented in gate-equivalents, which for this technology are equal to $0.199 \mu\text{m}^2$. Power numbers were extracted with Synopsys PrimeTime 2016.12, the switching activities obtained by a RTL simulation of the synthesized design in worst-case conditions².

4.2 Performance, power, and area results

We analyzed the AXI-CHI bridge by itself, trying to distinguish any performance limitations inherent to the design. We connected the bridge to an AXI master capable of generating 16 outstanding requests, their length chosen randomly accordingly to the burst length distribution of the AXI transactions generated by the DMA

²SSG/0.72 V/125 °C.

when calculating a 3×3 convolution [30]. We assume that this burst length distribution is representative of linear algebra kernels in general. Four instances of our bridge were analyzed, with a varying number n of AXI outstanding transactions it can handle (i.e., the number of protocol translators), while keeping the number of CHI outstanding transactions at its maximum, 256. This means that the ROB can hold 256 data flits. Table 2 presents the post-synthesis area and clock frequency results for these instances.

Table 2: Post-synthesis clock frequency and area results for several instances of the AXI-CHI bridge, with a varying number n of AXI outstanding transactions.

n	Clock [GHz]	Area [kGE]			
		Logic	Registers	Macros	Total
2	1.4	44.5	119.1	412.3	576.0
4	1.4	50.0	121.9	412.3	584.2
8	1.4	61.3	127.7	412.3	601.4
16	1.3	77.1	138.8	412.3	628.2

The clock frequencies are close to the limit imposed by the SRAM cuts. The retry mechanism, explained in Section 3.2, is the critical path, 31 logic levels deep. The area of the bridge is dominated by the ROB, which takes up to 72% of the total area. The number of protocol translators is a secondary parameter with small influence on area and timing. Excluding the area of the reorder buffer, the bridge has an area comparable to the area occupied by Ariane, which occupies 210 kGE at the same technology [33].

Figure 4 shows the throughput achieved by the bridge as a function of the memory latency. The bridge achieves up to 97% utilization of the CHI bus. The bridge with support for 16 AXI outstanding transactions has a lower steady-state throughput due to its lower operating frequency. Little’s Law [1] explains the steep performance drop with increasing memory latency: the 16 AXI outstanding transactions generated by the master occupy the bridge for $c = 81$ cycles, on average. A memory latency ℓ higher than this threshold will cause the bridge to starve during $\ell - c$ cycles, while the AXI master waits for the completion of an outstanding request before issuing another request. The bridge utilization is roughly inversely proportional to the memory latency, a direct consequence of Little’s Law. In spite of this performance degradation, the bridge is able to achieve a high throughput, up to 97% of the peak throughput of 512 bit/cycle, considering the instance able to handle 16 AXI outstanding transactions and a memory latency of 16 cycles.

At its peak throughput, in the worst-case corner, the bridge consumes up to 80.7 mW, 79% of which by the 1R1W SRAMs. Figure 5 shows the normalized energy consumption of our AXI-CHI bridge, in fJ/bit, as a function of the memory latency. The instance with support to 16 AXI outstanding transactions consumes as low as 125 fJ/bit for a moderate latency, e.g., the case where caches are serving the requests. An increasing memory latency decreases the bridge utilization, raising the energy consumption of the same instance to 173 fJ/bit for a memory latency of 256 cycles.

The previous results highlight the central point of the ROB in our design. At its largest—when it holds 256 data flits—the ROB amounts

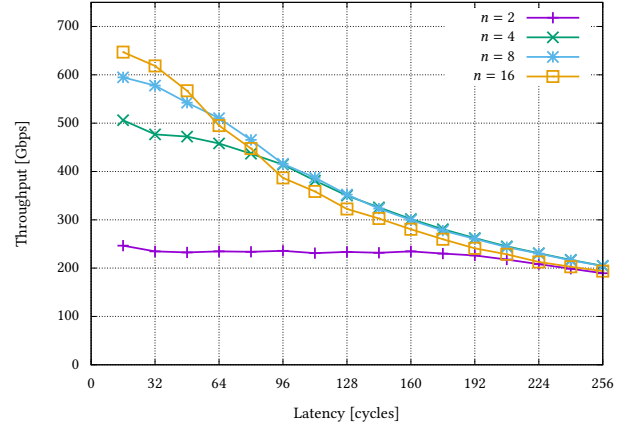


Figure 4: Throughput of the AXI-CHI bridge as a function of the memory latency, for a varying number of AXI outstanding transactions n .

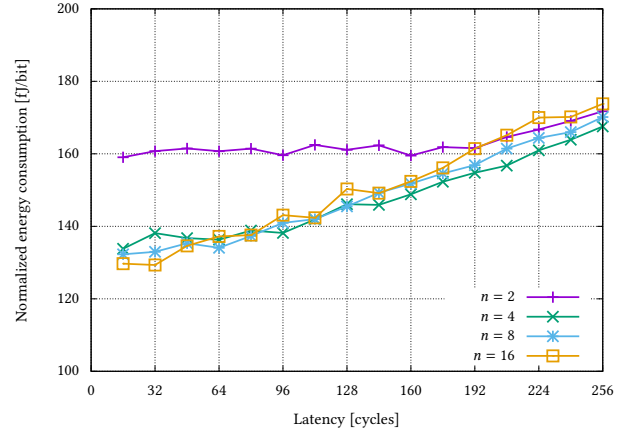


Figure 5: Normalized energy consumption of the AXI-CHI bridge, in fJ/bit, as a function of the memory latency, with a varying number n of AXI outstanding transactions.

to 70% of the area occupied by the bridge, and consumes 79% of its power. This sizeable ROB might be reduced with no impact on performance, granted that the memory latency is small enough. The ideal size of the ROB depends on how many outstanding AXI transactions the PMCA is able to request in parallel, and of the length of these AXI bursts.

4.3 Benchmark results

In this section, we analyze the performance achieved by the kernels of Table 1 with an increasing memory latency, as they run on the cluster described in Section 4.1. The AXI domain of the PULP cluster is bridged to a CHI memory plug via the bridge with support to 8 AXI outstanding transactions. Figure 6 shows the normalized runtime of the benchmarks listed in Table 1, each bar representing

a different memory latency ℓ . The runtime is relative to the baseline, where the main memory has eight cycles of latency.

All the kernels into consideration had a similar behavior, the normalized runtime being approximately proportional to the memory latency, following a behavior similar to the one shown in Figure 4. While these results were taken with fairly small matrices (e.g., 8×8 matrices for *gemm*), due to runtime constraints, it is possible that some of the kernels become partially latency-insensitive with larger data sizes, once the internal caches of the PMCA are being used more often than the external coherent interface.

Moreover, this order of magnitude for the main memory latency is to be expected at common heterogeneous systems. For example, Bansal et al. [5] report that the Dynamic Random-Access Memories (DRAMs) in the Programmable Logic (PL) of a Xilinx Field-Programmable Gate Array (FPGA) have a latency of about 200 ns, when accesses from high-performance ARM Cortex-A cores outside the FPGA cores. Therefore, a main memory latency on the hundredths of cycles is to be expected.

5 RELATED WORK

As innovation in computing shifts from single-core performance to manycore acceleration, heterogeneous data sharing tends to move into the spotlight. The vast increase in the number of interconnect technologies in recent years reflects this shift. In this section, we compare current technologies to bridge from non-coherent to coherent memory access and describe how our work applies to them. In general, our work is the first (to the best of our knowledge) that discusses the design and architecture of a hardware module that implements *any* of these technologies.

ACP, introduced in 2009 by ARM, is an optional 64-bit AXI3 slave port on Cortex-A9 CPUs [2, Section 2.4]. This port lets AXI-based accelerators access memory which is coherent to the caches of the CPU. ACP is optimized for bursts that are naturally aligned with cache lines and does not support coherent exclusive memory accesses [2, Section 2.4.2]. It offers accelerators up to 80% of the port's peak duplex bandwidth on cache hits, but less than 40% of the effective memory bandwidth on cache misses [29]. ACE5-LiteACP [4, Section E6.1] is a subset of ACE5-Lite introduced in 2018, superseding ACP. It only supports bursts of 128 or 512 bits, write-back cache accesses, non-snooping reads, and non-exclusive accesses. Our bridge supports a larger subset of CHI.

Coherent Accelerator Processor Interface (CAPI) [31], on its versions CAPI1 and CAPI2, is a proprietary bus standard by IBM to connect CPUs to GPUs and other external accelerators. It was initially designed in 2014, working over PCIe. CAPI is implemented on the CPU side by a Coherent Accelerator Processor Proxy (CAPP) and on the accelerator side by the Power Service Layer (PSL), which lets the accelerator become a cache peer to the other agents on the CAPI. OpenCAPI3 and OpenCAPI4 [25, 32] are open standards successor to CAPI, originally introduced in 2016 by AMD, Dell, Google, HP, IBM, Mellanox, Micron, NVIDIA, and Xilinx. OpenCAPI is layered on top of Bluelink 25G and does not require PSL at the accelerator side. With an adapted design on the coherent side, OpenCAPI can be another target for our bridge.

NVLink [12] is a proprietary protocol introduced 2014 by NVIDIA for data transfer between CPUs and GPUs and among GPUs. It

is currently supported by IBM Power CPUs and NVIDIA GPUs. NVLink 2.0 supports coherent accesses, significantly reducing access latency. A bandwidth efficiency of up to 88.9% is reported when using the “copy engine” of the GPU together with dedicated NVSwitch chips [24]. NVLink is a closed specification. Therefore, we are not able to comment on the compatibility of our bridge with this protocol, although we do achieve a significantly higher bandwidth efficiency in coherent on-chip communication.

Gen-Z [16] is an open interconnect specification released in 2018 by AMD, ARM, Dell, HP, IBM, Huawei, Mellanox, and Xilinx. Gen-Z transactions are focused on loads and stores issued by CPU cores today. While Gen-Z seems promising for making various current memory technologies—such as DRAM, NAND Flash, and Storage Class Memory (SCM)—available to CPUs and accelerators alike, its applicability to burst-based accelerator memory accesses still remains to be shown.

The Coherent Interconnect for Accelerator (CCIX) [8] is an open chip-to-chip interconnect standard released in 2018 by AMD, ARM, Huawei, Mellanox, Qualcomm, and Xilinx. It consists of the specification of a protocol and transaction layer working on top of the PCIe data link layer. CCIX supports AMBA CHI, among other memory coherent protocols, on its protocol layer. In that sense, our bridge is also compatible with CCIX.

Finally, the Compute Express Link (CXL) [10] is an open interconnect standard introduced in 2019 by AMD, ARM, Dell, Google, HP, Huawei, Intel, Mellanox, and NVIDIA. CXL was designed as a high-bandwidth, low-latency interconnect between accelerators and CPUs. It consists of a transaction and data link layer on top of the physical layer of PCIe 5.0. CXL allows accelerators to coherently access CPU memory (CXL.cache) and CPUs to access accelerator memory (CXL.memory), while the CPU keeps maintaining coherency. As with OpenCAPI, CXL.cache could be used as another target for our bridge.

6 CONCLUSIONS

We have presented the architecture of a bridge between burst-based non-coherent AXI transfers generated by a PMCA—here, an open-source PULP cluster with eight 32-bit RISC-V cores and 256 KiB of L1 SPM—with a coherent memory subsystem based on the CHI protocol. We used open-source readily available designs throughout the development of our bridge, using Ariane, a 64-bit RISC-V core, as the host processor, and HERO as our heterogeneous platform. The design of our AXI-CHI bridge will also be open-sourced under a permissive license.

We showed post-synthesis results of several instances of our bridge in GLOBALFOUNDRIES 22FDX technology, which has an energy consumption down to 130 fJ/bit at 1.3 GHz, at worst-case conditions. Both the area and the power of the bridge are dominated by a Re-order Buffer (ROB), required to reconstruct the order of the data beats of an AXI burst, which might come back out of order from the CHI interface. It is possible to have a smaller ROB, granted that the bridge is used with a memory system with equivalently smaller memory latency.

We measured the performance achieved by the PMCA running some common linear algebra kernels from the Polybench/ACC suite, as a function of the memory latency. We found out that

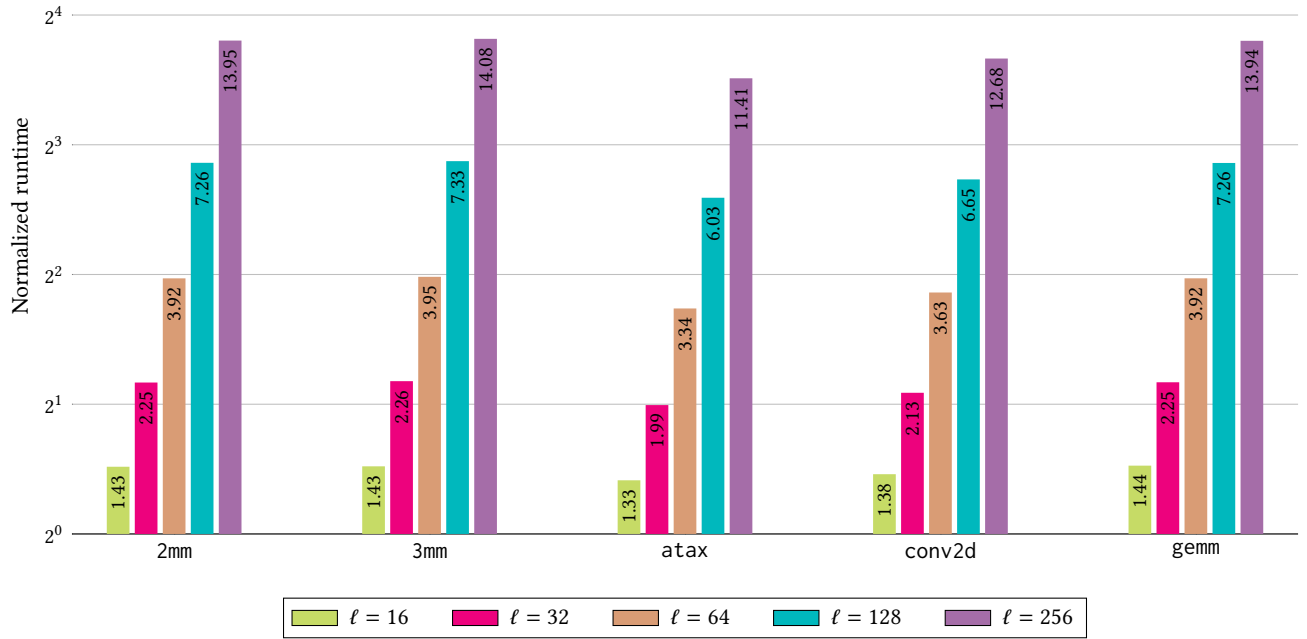


Figure 6: Normalized performance of selected Polybench/ACC benchmarks as a function of the memory latency ℓ .

even compute intensive kernels suffered a performance degradation with an increasing memory latency. This is an artifact of crossing the non-coherent burst-based domain towards a coherent cache-line-based memory system. In fact, the memory latency is a major constraint on the performance achieved by the bridge. This is due to a protocol-level constraint of CHI, not present at AXI. CHI requires that the transaction IDs of all active transactions generated by a request node must be unique at all times. There are at most 256 transaction IDs, and every CHI transaction corresponds to a single data flit. Once a Requester exhausts its transaction IDs, it must wait for the end of an active transaction until it can issue the next request. Therefore, CHI effectively limits, at a protocol level, the main memory latency that a Request node can tolerate.

ACKNOWLEDGMENTS

The authors would like to thank Florian Zaruba and Frank Gürkaynak for the discussions and insights.

REFERENCES

- [1] Arnold O. Allen. 1990. *Probability, Statistics, and Queueing Theory with Computer Science Applications*. Academic Press Professional, Inc., San Diego, CA, USA.
- [2] ARM. 2009. *Arm Cortex-A9 MPCore Technical Reference Manual*. ARM. ARM DDI 0407E.
- [3] ARM. 2018. *AMBA 5 CHI Architecture Specification*. ARM, Cambridge, England. ARM IHI 0050C.
- [4] ARM. 2019. *AMBA AXI and ACE*. ARM, Cambridge, England. ARM IHI 0022G.
- [5] Giovanni Gracioli Renato Mancuso Rodolfo Pellizzoni Ayoosh Bansal, Rohan Tabish and Marco Caccamo. 2018. Evaluating the memory subsystem of a configurable heterogeneous MPSoC. In *Workshop on Operating Systems Platforms for Embedded Real-Time Applications (OSPert)*. Barcelona, Spain, 55.
- [6] T. B. Berg. 2009. Maintaining IO Data Coherence in Embedded Multicore Systems. *IEEE Micro* 29, 3 (May 2009), 10–19. <https://doi.org/10.1109/MM.2009.44>
- [7] A. Capotondi, A. Marongiu, and L. Benini. 2018. Runtime Support for Multiple Offload-Based Programming Models on Clustered Manycore Accelerators. *IEEE Transactions on Emerging Topics in Computing* 6, 3 (July 2018), 330–342. <https://doi.org/10.1109/TETC.2016.2554318>
- [8] CCIX Consortium. 2018. *CCIX Base Specification 1.0*. CCIX Consortium. <https://www.ccixconsortium.com/library/specification/>
- [9] Cray Inc. 2019. *Cray XC Series*. <https://www.cray.com/sites/default/files/Cray-XC-Series-Brochure.pdf>
- [10] CXL Consortium. 2019. *CXL 1.1 Specification*. CXL Consortium. <https://www.computexpresslink.org/resource-library>
- [11] William Dally and Brian Towles. 2003. *Principles and Practices of Interconnection Networks*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [12] D. Foley and J. Danskin. 2017. Ultra-Performance Pascal GPU and NVLink Interconnect. *IEEE Micro* 37, 2 (March 2017), 7–17. <https://doi.org/10.1109/MM.2017.37>
- [13] Andrei Frumusanu. 2018. The Apple A12 – First Commercial 7nm Silicon. <https://www.anandtech.com/show/13392/the-iphone-xs-xs-max-review-unveiling-the-silicon-secrets/2>
- [14] Andrei Frumusanu. 2019. Samsung Announces the Exynos 9825 SoC: First 7nm EUV Silicon Chip. <https://www.anandtech.com/show/14645/samsung-announces-exynos-9825-first-7nm-euv-silicon-chip>
- [15] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini. 2017. Near-Threshold RISC-V Core With DSP Extensions for Scalable IoT Endpoint Devices. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 10 (Oct. 2017), 2700–2713. <https://doi.org/10.1109/TVLSI.2017.2654506>
- [16] Gen-Z Consortium. 2018. *Gen-Z Specification 1.0*. Gen-Z Consortium. <https://genzconsortium.org/specifications/>
- [17] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos. 2012. Auto-tuning a high-level language targeted to GPU codes. In *2012 Innovative Parallel Computing (InPar)*. 1–10. <https://doi.org/10.1109/InPar.2012.6339595>
- [18] Intel Corp. 2016. IvyTown Xeon + FPGAs: The HARP Program. In *Proceedings of the 43rd ACM/IEEE International Symposium on Computer Architecture (ISCA)*. ACM. https://cpufpga.files.wordpress.com/2016/04/harp_isca_2016_final.pdf
- [19] Andreas Kurth, Pirmin Vogel, Alessandro Capotondi, Andrea Marongiu, and Luca Benini. 2017. HERO: Heterogeneous Embedded Research Platform for

- Exploring RISC-V Manycore Accelerators on FPGA. In *Proceedings of Computer Architecture Research with RISC-V Workshop (CARRV '17)*. Boston, MA. <https://doi.org/10.3929/ethz-b-000219249>
- [20] Andreas Kurth, Pirmin Vogel, Andrea Marongiu, and Luca Benini. 2018. Scalable and Efficient Virtual Memory Sharing in Heterogeneous SoCs with TLB Prefetching and MMU-Aware DMA Engine. In *2018 IEEE 36th International Conference on Computer Design (ICCD)*. IEEE, 292–300. <https://doi.org/10.1109/ICCD.2018.00052>
- [21] Andreas Kurth, Koen Wolters, Björn Forsberg, Alessandro Capotondi, Andrea Marongiu, Tobias Grosser, and Luca Benini. 2020. Mixed-Data-Model Heterogeneous Compilation and OpenMP Offloading. In *Proceedings of the 29th International Conference on Compiler Construction (CC 2020)*. Association for Computing Machinery, New York, NY, USA, 119–131. <https://doi.org/10.1145/3377555.3377891>
- [22] NVIDIA. 2015. NVIDIA Tegra X1. <https://international.download.nvidia.com/pdf/tegra/Tegra-X1-whitepaper-v1.0.pdf>
- [23] NVIDIA. 2017. *NVIDIA Tesla V100 GPU Architecture*. NVIDIA. <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf> v1.1.
- [24] NVIDIA. 2018. NVSwitch and DGX-2: NVLink Switching Chip and Scale-Up Compute Server. IEEE HotChips 30. https://www.hotchips.org/hc30/2conf/2.01_Nvidia_NVswitch_HotChips2018_DGX2NVS_Final.pdf
- [25] OpenCAPI Consortium 2016. *OpenCAPI 4.0 Specifications*. OpenCAPI Consortium. <https://opencapi.org/technical/specifications/>
- [26] D.A. Patterson and J.L. Hennessy. 2017. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Elsevier Science, Cambridge, MA.
- [27] Qualcomm Inc. 2019. Qualcomm Snapdragon 855 mobile platform product brief. <https://en.wikichip.org/w/images/3/38/snapdragon-855-mobile-platform-product-brief.pdf>
- [28] D. Rossi, F. Conti, A. Marongiu, A. Pullini, I. Loi, M. Gautschi, G. Tagliavini, A. Capotondi, P. Flatresse, and L. Benini. 2015. PULP: A parallel ultra low power platform for next generation IoT applications. In *2015 IEEE Hot Chips 27 Symposium (HCS)*. 1–39. <https://doi.org/10.1109/HOTCHIPS.2015.7477325>
- [29] Mohammadsadegh Sadri, Christian Weis, Norbert Wehn, and Luca Benini. 2013. Energy and Performance Exploration of Accelerator Coherency Port Using Xilinx ZYNQ. In *Proceedings of the 10th FPGAWorld Conference (FPGAWorld '13)*. ACM, New York, NY, USA. <https://doi.org/10.1145/2513683.2513688>
- [30] Fabian Schuiki, Michael Schaffner, Frank Gürkaynak, and Luca Benini. 2018. A Scalable Near-Memory Architecture for Training Deep Neural Networks on Large In-Memory Datasets. *IEEE Trans. Comput.* (2018), 1–1. <https://doi.org/10.1109/TC.2018.2876312>
- [31] J. Stuecheli, B. Blaner, C. R. Johns, and M. S. Siegel. 2015. CAPI: A Coherent Accelerator Processor Interface. *IBM Journal of Research and Development* 59, 1 (Jan. 2015), 7:1–7:7. <https://doi.org/10.1147/JRD.2014.2380198>
- [32] J. Stuecheli, W. J. Starke, J. D. Irish, L. B. Arimilli, D. Dreps, B. Blaner, C. Wollbrink, and B. Allison. 2018. IBM POWER9 opens up a new era of acceleration enablement: OpenCAPI. *IBM Journal of Research and Development* 62, 4/5 (July 2018), 8:1–8:8. <https://doi.org/10.1147/JRD.2018.2856978>
- [33] F. Zaruba and L. Benini. 2019. The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (Nov. 2019), 2629–2640. <https://doi.org/10.1109/TVLSI.2019.2926114>